Monte Carlo and Sarsa(λ) for Text Flappy Bird

KAPOTOS Fotios

CentraleSupélec
fotiskapotos@gmail.com

Abstract. We apply tabular reinforcement learning to *TextFlappyBird-v0*, comparing a first-visit Monte Carlo (MC) agent with a Sarsa(λ) agent over a compact 644-entry Q-table derived from the 2-D distance observation. Both algorithms are tuned via a hyperparameter grid search using a fair exploration-fraction parameterisation, then retrained for 5 000 episodes. Both agents achieve perfect play on the training configuration. MC converges reliably across hyperparameters (9/18 sweep configurations reach the reward ceiling), while Sarsa(λ) is more sensitive but matches MC with optimal settings. In zero-shot transfer experiments, MC degrades sharply when the environment grows larger than the training grid, while Sarsa(λ) retains substantially higher performance, suggesting that online TD updates with eligibility traces produce more spatially robust Q-values.

1 Environment and Experimental Setup

TextFlappyBird-v0 [2] provides a discretised Flappy Bird variant whose observation is $(x_{\text{dist}}, y_{\text{dist}}) \in \mathbb{Z}^2$: horizontal distance to the next pipe and signed vertical offset from the gap centre. The action space is binary (flap / no-flap); reward is $+1$ per step survived, -1 on collision. For the baseline configuration (height=15, width=20, pipe_gap=4) the reachable space has $x_{\max} = 13$, $y_{\max} = 11$, giving a Q-table of $(14 \times 23 \times 2) = 644$ entries — small enough to learn exhaustively within a few thousand episodes.

Environment versions. *TextFlappyBird-v0* returns the compact 2-D vector above. *TextFlappyBird-screen-v0* returns the full character-based screen render, an (height $\times$ width) array that grows with the grid, making tabular Q-learning impractical and requiring function approximation. The distance version abstracts away visual complexity but assumes direct pipe-distance observability, which does not hold in the real game.

Original Flappy Bird Gym [3]. The Talendar implementation exposes continuous pixel observations (288$\times$512 RGB). Tabular methods cannot handle this infinite state space directly; the same algorithmic principles would require function approximation (e.g. deep MC or deep Sarsa with a neural network).

2 Methods

2.1 Monte Carlo Control (first-visit)

The MC agent [1] maintains a Q-table and updates it at episode end using discounted returns walked in reverse:

$$G_t = r_t + \gamma G_{t+1}, \quad Q(s,a) \leftarrow \text{mean}\big(\{G : (s,a) \text{ first-visited}\}\big).$$

The policy is ε-greedy; ε decays from 1 to 0.01. Out-of-range observations are clipped to the nearest valid Q-table index.

2.2 Sarsa(λ)

Sarsa(λ) [1] performs online updates with eligibility traces. At each step: $\delta = r + \gamma Q(s',a') - Q(s,a)$, $e(s,a) \mathrel{+}= 1$, $Q \mathrel{+}= \alpha\delta e$, $e \mathrel{\times}= \gamma\lambda$. $\lambda = 0$ reduces to one-step TD; $\lambda \to 1$ approaches MC.

2.3 Hyperparameter Search and Final Training

To ensure fair comparison across a short 1 000-episode sweep, both searches parameterise exploration by *fraction of training* rather than a raw per-episode decay: $\varepsilon_{\text{decay}} = (\varepsilon_0 - \varepsilon_{\min})/(f \cdot N_{\text{sweep}})$, so every configuration reaches $\varepsilon_{\min}$ at the same relative point. The retrain decay is scaled to the full 5 000-episode run: $\varepsilon^{\text{full}}_{\text{decay}} = (\varepsilon_0 - \varepsilon_{\min})/(f \cdot N_{\text{full}})$.

- **MC:** $\gamma \in \{0.85, 0.90, 0.95\}$, $f \in \{0.3, 0.6, 0.9\}$, mode $\in \{\text{linear}, \text{exp}\}$.
- **Sarsa(λ):** $\alpha \in \{0.05, 0.10, 0.20\}$, $\lambda \in \{0.70, 0.90, 0.95\}$, $f \in \{0.3, 0.6, 0.9\}$.

3 Results

3.1 Hyperparameter Sensitivity and Best Configurations

Table 1 summarises the best configurations found. MC is markedly more robust: 9 of 18 sweep configurations reach the reward ceiling (3 000, equal to the episode step limit), versus only 2 of 27 for Sarsa(λ). MC's best setting ($\gamma = 0.85$, exponential decay, $f = 0.3$) uses fast exploration (the policy becomes greedy after only 30% of training) which is sufficient because the Q-table is fully covered quickly. Sarsa's best setting ($\alpha = 0.1$, $\lambda = 0.7$, $f = 0.9$) requires slow exploration, staying ε-greedy for 90% of training. Notably, the optimal $\lambda = 0.7$ is lower than the common default of 0.9, indicating that shorter eligibility traces (closer to one-step TD) are preferable on this episodic task.

3.2 Learning Curves and Value Functions

Figure 1 shows training curves and final evaluation for both best-tuned agents; both reach the 3 000 reward ceiling, confirming perfect play. Figure 2 shows the state-value function $V(s) = \max_a Q(s, a)$ and greedy policy. Both agents learn the correct structure: flap when below the gap centre ($y_{\text{dist}} > 0$), coast otherwise. The MC value function is sharper (exact sample-mean updates), while Sarsa produces smoother value gradients due to bootstrapping.

4 Adaptability to Environment Variations

Trained agents are evaluated zero-shot on modified environments; states outside the trained Q-table range are clipped (Fig. 3).

Pipe gap. Both agents achieve perfect scores for gap$\geq$4 and fail below it (gap=2: MC$\approx$55, Sarsa$\approx$28; gap=3: MC$\approx$90, Sarsa$\approx$24). The clipped states cannot represent the narrower critical region, and neither agent has learned to navigate it.

Height. Both agents remain perfect for height$\leq$15 (same or smaller $y_{\max}$). For larger heights, Sarsa generalises dramatically better: at height=18, MC drops to 398 ± 435 while Sarsa scores 2611 ± 1008; at height=20, MC reaches only 188 ± 215 versus Sarsa's 1121 ± 979.

Width. At the training width (20) both are perfect; for width=25, MC scores 217 ± 204 versus Sarsa's 1461 ± 1193. Sarsa also handles the narrower width=15 better (2221 ± 1005 vs 1668 ± 1059).

5 Discussion

MC robustness vs. Sarsa sensitivity. MC's sample-mean update is unbiased regardless of episode length or ε level: any trajectory produces a valid return estimate. Sarsa bootstraps from $Q(s', a')$, which is noisy early in training, creating high-variance TD errors that destabilise learning under suboptimal hyperparameters. This explains why only $2/27$ Sarsa configurations reach the optimum in the short sweep, versus $9/18$ for MC. Once properly tuned (lower $\alpha = 0.1$, $\lambda = 0.7$, long exploration), Sarsa matches MC on the training task.

Sarsa's superior transfer to larger environments. The most striking result is Sarsa's advantage when the environment is *larger* than the training grid. When height or width grows, new out-of-range states are clipped to the Q-table boundary. Sarsa's eligibility traces propagate credit along entire trajectories, ensuring that boundary-adjacent states receive consistent updates throughout training.

Limitations and future work. The 2-D state abstraction discards absolute bird position and velocity, limiting multi-step planning. Scaling to the screen-based or continuous observation spaces requires function approximation (e.g. neural network Q-learning), where the convergence properties of MC and Sarsa differ significantly from the tabular case.

References

1. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge, MA (2018)
2. Christodoulidis, S.: Text Flappy Bird Gym. https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym, last accessed 2026/04/05
3. Talendar: Flappy Bird Gym. https://github.com/Talendar/flappy-bird-gym, last accessed 2026/04/05

A Figures and Tables

Table 1. Best hyperparameters and final greedy evaluation (100 episodes, max steps = 3 000 per episode).

	MC	**Sarsa(λ)**
Best γ	0.85	0.99 (fixed)
Best α	—	0.10
Best λ	—	0.70
Explore frac f	0.30	0.90
Decay mode	exponential	linear
Sweep configs optimal	9/18	2/27
Final eval mean reward	3 000	3 000

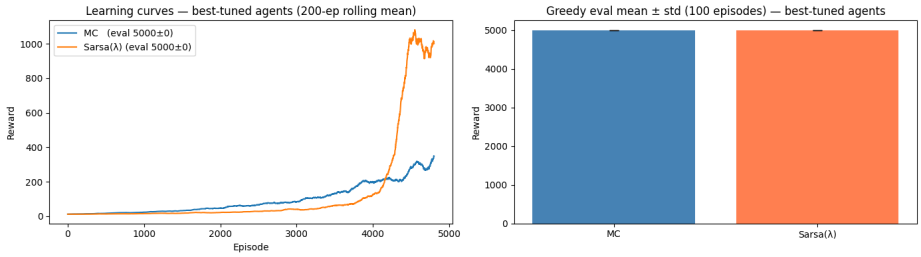

Fig. 1. Learning curves (200-ep rolling mean) and greedy evaluation (mean±std, 100 episodes) for the best-tuned agents.

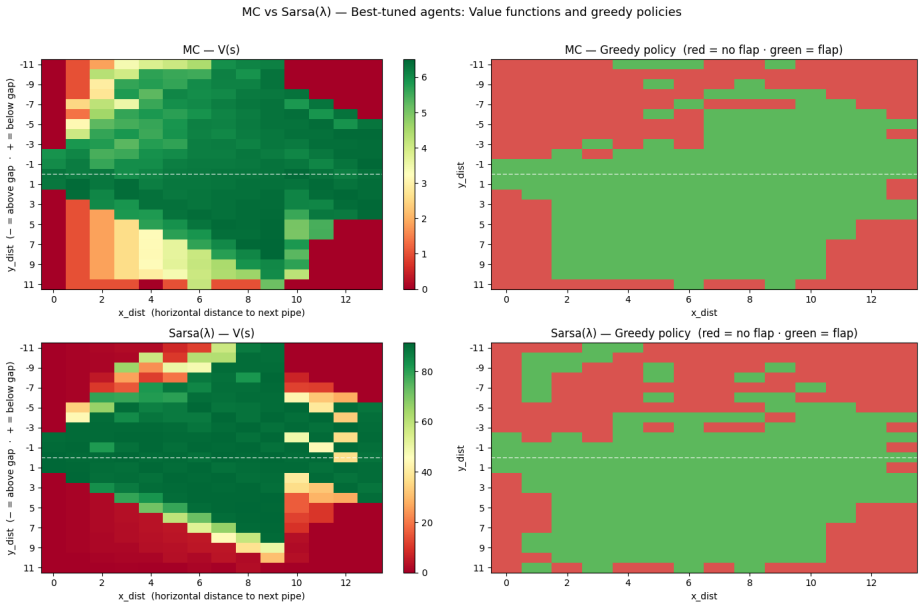

Fig. 2. $V(s)$ (left) and greedy policy (right) for MC (top) and Sarsa(λ) (bottom). Green = flap, red = no-flap. Dashed line: gap centre ($y_{\text{dist}} = 0$).

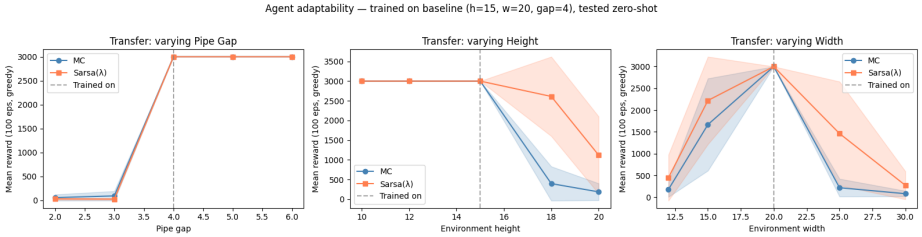

Fig. 3. Zero-shot transfer: mean reward (±1 std, 100 episodes) vs pipe gap (left), height (centre), and width (right). Vertical dashed line: training configuration.